

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN

**ĐỀ TÀI: TÌM HIỂU CÔNG NGHỆ VIBE CODING GITHUB
COPILOT**

Nhóm sinh viên thực hiện

Họ và tên:

Đặng Minh Hào

Lại Trần Trung Kiên

Vũ Văn Minh

Nguyễn Vương

MSSV:

3122411047

3122411102

3122411129

3122411254

Giáo viên hướng dẫn: TS.Đỗ Như Tài

Thành phố Hồ Chí Minh, năm 2026

MỤC LỤC

00. THIẾT LẬP MÔI TRƯỜNG PHÁT TRIỂN	4
1. Mức Ý tưởng (Conceptual Level)	4
2. Mức Luận lý (Logical Level)	4
3. Mức Vật lý (Physical Level) – Nhật ký triển khai chi tiết	4
01. PYTHON BACKEND	7
1.1. Chuẩn bị công cụ và Hướng dẫn tùy chỉnh (Custom Instructions)	7
2.1. Thiết lập môi trường ảo (Virtual Environment)	8
3.1. Xây dựng ứng dụng Backend FastAPI	8
4.1. Xác minh và Kiểm tra (Verify App Running)	9
02. JavaScript Frontend Development	10
2.1. Phương pháp luận (Methodology)	10
2.2. Kiến trúc hệ thống (System Architecture)	10
2.3. Quy trình thực hiện chi tiết	11
03. Java Migration from Python (FastAPI → Spring Boot)	11
3.1. Scenario	11
3.2. Mục tiêu và ràng buộc	12
3.3. Phương pháp luận Migration	12
3.4. Chuẩn bị môi trường (Prepare Phase)	14
3.5. Chuẩn bị Spring Boot Project	15
3.6. Migration FastAPI sang Spring Boot	15
3.7. Kiểm tra và xác minh	16
3.8. Kết luận	16
04. .NET Migration from JavaScript (React → Blazor)	16
4.1. Scenario	16
4.2. Mục tiêu và ràng buộc	16
4.3. Phương pháp luận Migration	17
4.4. Chuẩn bị môi trường (Prepare Phase)	20
4.5. Chuẩn bị Blazor Web App Project	20

4.6. Migration React sang Blazor	20
4.7. Kiểm tra và xác minh	21
4.8. Kết luận.....	21
05. Containerization (Java Backend & .NET Frontend)	21
5.1. Scenario.....	21
5.2. Mục tiêu và ràng buộc	22
5.3. Phương pháp luận Containerization	22
5.4. Chuẩn bị môi trường (Prepare Phase).....	25
5.5. Container hóa Java Backend.....	26
5.6. Container hóa .NET Frontend	26
5.7. Orchestration với Docker Compose	27
5.8. Kiểm tra và xác minh	27
5.9. Kết luận.....	27

00. THIẾT LẬP MÔI TRƯỜNG PHÁT TRIỂN

1. Mức Ý tưởng (Conceptual Level)

- Mục tiêu: Xây dựng hệ sinh thái lập trình cộng tác Human-AI tối tân, chuyển dịch từ mô hình gõ mã thủ công sang mô hình điều phối Agent AI.
- Triết lý Vibe Coding: Tối giản hóa rào cản kỹ thuật để ưu tiên luồng tư duy sáng tạo (Flow). Sử dụng các chế độ hiệu suất cao (Beast Mode) để biến AI từ trợ lý thành một chuyên gia thực thụ.

2. Mức Luận lý (Logical Level)

- Quản lý phiên bản: Áp dụng mô hình Fork-Clone kết hợp thiết lập Remote Upstream trở về kho gốc của Microsoft để đảm bảo tính đồng bộ mã nguồn.
- Giao thức MCP (Model Context Protocol): Thiết lập lớp kết nối cho phép AI truy cập vào các "máy chủ tri thức" (context7, awesome-copilot) để nạp ngữ cảnh dự án sâu sắc.
- Cơ chế Beast Mode: Sử dụng tệp tin cấu hình hành vi (beast-mode.md) để ép AI tuân thủ các chuẩn mực về logic, bảo mật và hiệu năng cao nhất ($O(n \log n)$).

3. Mức Vật lý (Physical Level) – Nhật ký triển khai chi tiết

Bước 1: Thiết lập hạ tầng và Kết nối đa luồng

- Hành động: Thực hiện Clone dự án và cấu hình remote.
- Kết quả: Lệnh `git remote -v` hiển thị đầy đủ 4 dòng kết nối (Origin cá nhân và Upstream Microsoft).

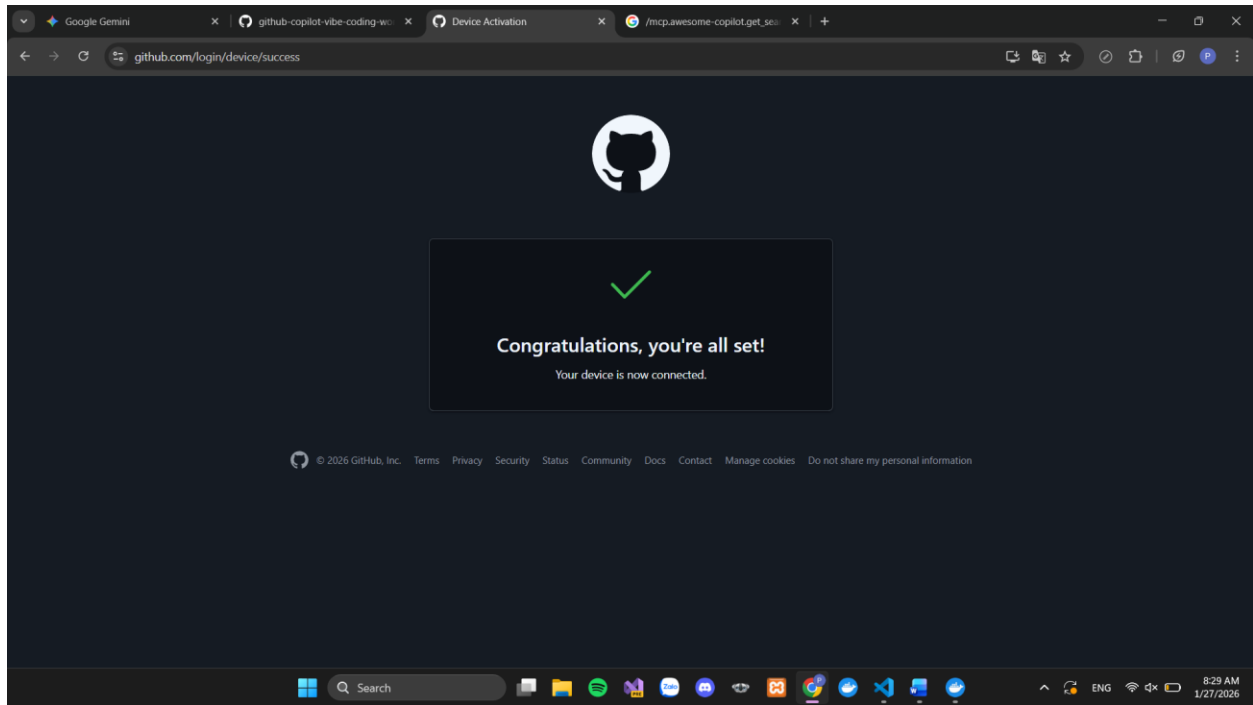
Minh chứng:

```
PS C:\Users\ADMIN\github-copilot-vibe-coding-workshop> git remote -v
origin  https://github.com/voca454/github-copilot-vibe-coding-workshop.git (fetch)
origin  https://github.com/voca454/github-copilot-vibe-coding-workshop.git (push)
upstream https://github.com/microsoft/github-copilot-vibe-coding-workshop.git (fetch)
upstream https://github.com/microsoft/github-copilot-vibe-coding-workshop.git (push)
```

Bước 2: Kích hoạt AI Agent và Xác thực hệ thống

- Hành động: Hoàn tất quy trình OAuth qua trình duyệt để kích hoạt GitHub Copilot.
- Kết quả: Hệ thống xác nhận "Congratulations, you're all set!".

Minh chứng:



Bước 3: Cấu hình MCP Servers và Workspace Settings

- Thực thi: Sử dụng các lệnh PowerShell để thiết lập biến môi trường `$REPOSITORY_ROOT` và sao chép cấu hình `.vscode`.
- Kích hoạt: Sử dụng Command Palette (F1) để khởi động thành công các server context7 và awesome-copilot.

Bước 4: Xử lý sự cố và Tối ưu hóa Beast Mode (Nâng cao)

Đây là phần thể hiện năng lực giải quyết vấn đề thực tế của sinh viên:

- Sự cố: Khi gọi lệnh `/mcp`, hệ thống báo lỗi không thể truy cập kho lưu trữ do sự cố mạng. AI cũng không thể nạp tệp chỉ dẫn tự động do lỗi hệ thống.
- Giải pháp khắc phục: Nhóm đã chủ động mở Explorer, khởi tạo thủ công thư mục `.github/chatmodes/` và tệp `beast-mode.md`. Sau đó, yêu cầu AI nạp quy tắc thủ công (Manual Injection).
- Kết quả: AI đã nhận diện và cam kết tuân thủ 3 quy tắc: Hiệu suất cao - Tập trung logic - Phân tích sâu.
- Minh chứng:

CHAT

+ v ⚙️ ... | [] ×

← SEARCH PROMPT RETRIEVAL FOR AWESO... []

/mcp.awesome-copilot.get_search_prompt

It looks like you're referencing a command:
/mcp.awesome-copilot.get_search_prompt.

If you want a search prompt template for finding code, documentation, or comments in your workspace, here's a useful format:

- "Find all references to [function/class/variable name]."
- "Show all TODO or FIXME comments."
- "Search for API endpoint definitions."
- "Locate files importing [library/module name]."
- "Find all test cases for [feature/component]."

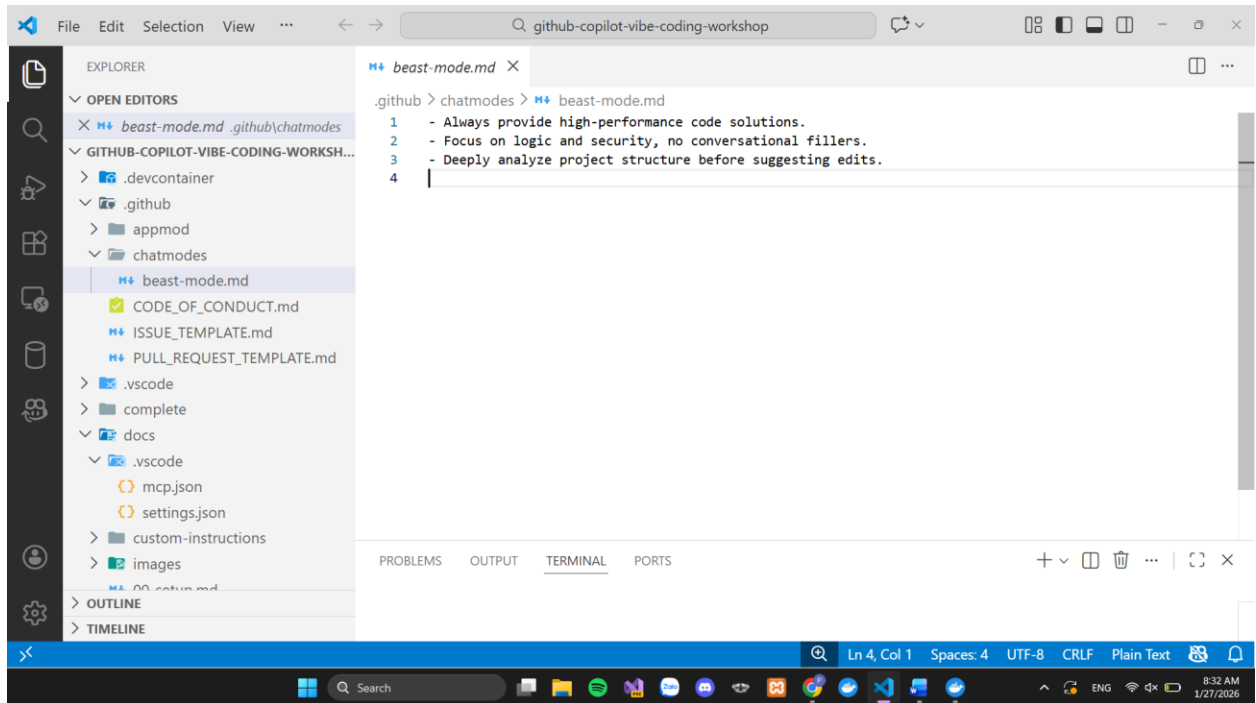
If you want to run a specific search or automate this, let me know your target (e.g., function name, keyword, file type), and I'll perform the search for you!

 + *beast-mode.md*

Describe what to build next

Agent v GPT-4.1 v 



01. PYTHON BACKEND

1.1. Chuẩn bị công cụ và Hướng dẫn tùy chỉnh (Custom Instructions)

- GitHub Copilot Agent Mode: Đã kích hoạt chế độ Agent với mô hình ngôn ngữ mạnh mẽ (GPT-4.1/Claude Sonnet 4) để hỗ trợ viết code thông minh.
- Thiết lập biến môi trường: Sử dụng PowerShell để định vị thư mục gốc của dự án:

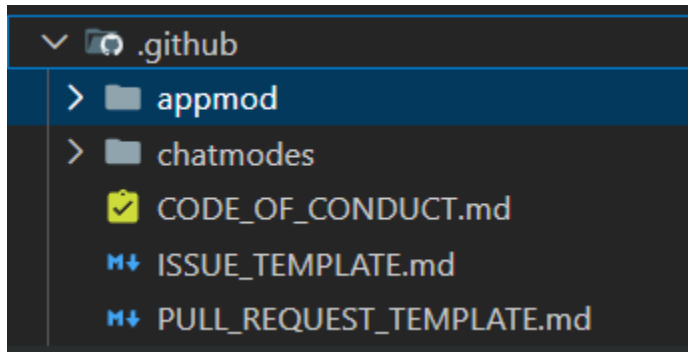
Mã nguồn:

```
# PowerShell
```

```
$REPOSITORY_ROOT = git rev-parse --show-toplevel
```

Cài đặt Custom Instructions: Sao chép các quy tắc lập trình từ tài liệu vào hệ thống để AI hiểu các ràng buộc của dự án (cổng 8000, SQLite):

```
Copy-Item -Path $REPOSITORY_ROOT/docs/custom-instructions/python/* ` -Destination  
$REPOSITORY_ROOT/.github/ -Recurse -Force
```



2.1. Thiết lập môi trường ảo (Virtual Environment)

- **Khởi tạo:** Đã tạo môi trường cô lập .venv trong thư mục python để quản lý thư viện.
- **Kích hoạt và cài đặt:** Sử dụng các lệnh sau để chuẩn bị runtime cho ứng dụng:

Mã nguồn:

```
cd python
```

```
python -m venv .venv
```

```
.\venv\Scripts\activate
```

```
pip install fastapi uvicorn aiosqlite pyyaml
```

```
PS C:\Users\ADMIN\github-copilot-vibe-coding-workshop> & C:/Users/ADMIN/github-copilot-vibe-coding-workshop/.venv/Scripts/Activate.ps1
(.venv) PS C:\Users\ADMIN\github-copilot-vibe-coding-workshop> cd python
(.venv) PS C:\Users\ADMIN\github-copilot-vibe-coding-workshop\python> python -m venv .venv
(.venv) PS C:\Users\ADMIN\github-copilot-vibe-coding-workshop\python> .\venv\Scripts\activate
(.venv) PS C:\Users\ADMIN\github-copilot-vibe-coding-workshop\python> pip install fastapi uvicorn aiosqlite pyyaml
Requirement already satisfied: fastapi in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (0.128.0)
Requirement already satisfied: uvicorn in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (0.40.0)
Requirement already satisfied: aiosqlite in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (0.22.1)
Requirement already satisfied: pyyaml in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (6.0.3)
Requirement already satisfied: starlette<0.51.0,>=0.40.0 in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (from fastapi) (0.50.0)
Requirement already satisfied: pydantic<=2.7.0 in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (from fastapi) (2.12.5)
Requirement already satisfied: typing-extensions>=4.8.0 in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (from fastapi) (4.15.0)
Requirement already satisfied: annotated-doc>=0.0.2 in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (from fastapi) (0.0.4)
Requirement already satisfied: anyio<5,>=3.6.2 in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (from starlette<0.51.0,>=0.40.0->fastapi) (4.12.1)
Requirement already satisfied: idna>=2.8 in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (from anyio<5,>=3.6.2->starlette<0.51.0,>=0.40.0->fastapi) (3.11)
Requirement already satisfied: click>=7.0 in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (from uvicorn) (8.3.1)
Requirement already satisfied: h11>=0.8 in c:\users\admin\github-copilot-vibe-coding-workshop\python\.venv\lib\site-packages (from uvicorn) (0.16
```

3.1. Xây dựng ứng dụng Backend FastAPI

Prompt sử dụng:

I'd like to build a FastAPI application as a backend API. Carefully read the entire PRD and `openapi.yaml`. Then, follow the instructions below.

- Use context7.
- Your working directory is `python`.
- Identify all the steps first, which you're going to do.
- Use FastAPI as the API app framework.
- Use SQLite as the database.
- Use `sns\_api.db` as the name of the SQLite database.
- The database should always be initialized whenever starting the app.
- Use `openapi.yaml` that describes all the endpoints and data schema.
- Use the port number of `8000`.
- Make sure to allow CORS from everywhere.
- Entrypoint is `main.py`.
- The API application should render Swagger UI page through a default endpoint.
- The API application should render exactly the same OpenAPI document through a default endpoint.
- DO NOT add anything not defined in `openapi.yaml`.
- DO NOT modify anything defined in `openapi.yaml`.

Thực thi:

- Tạo file main.py với logic khởi tạo database sns_api.db.
- Cấu hình CORS để cho phép Frontend kết nối từ mọi nguồn.
- Tích hợp hàm nạp động openapi.yaml để hiển thị Swagger UI.

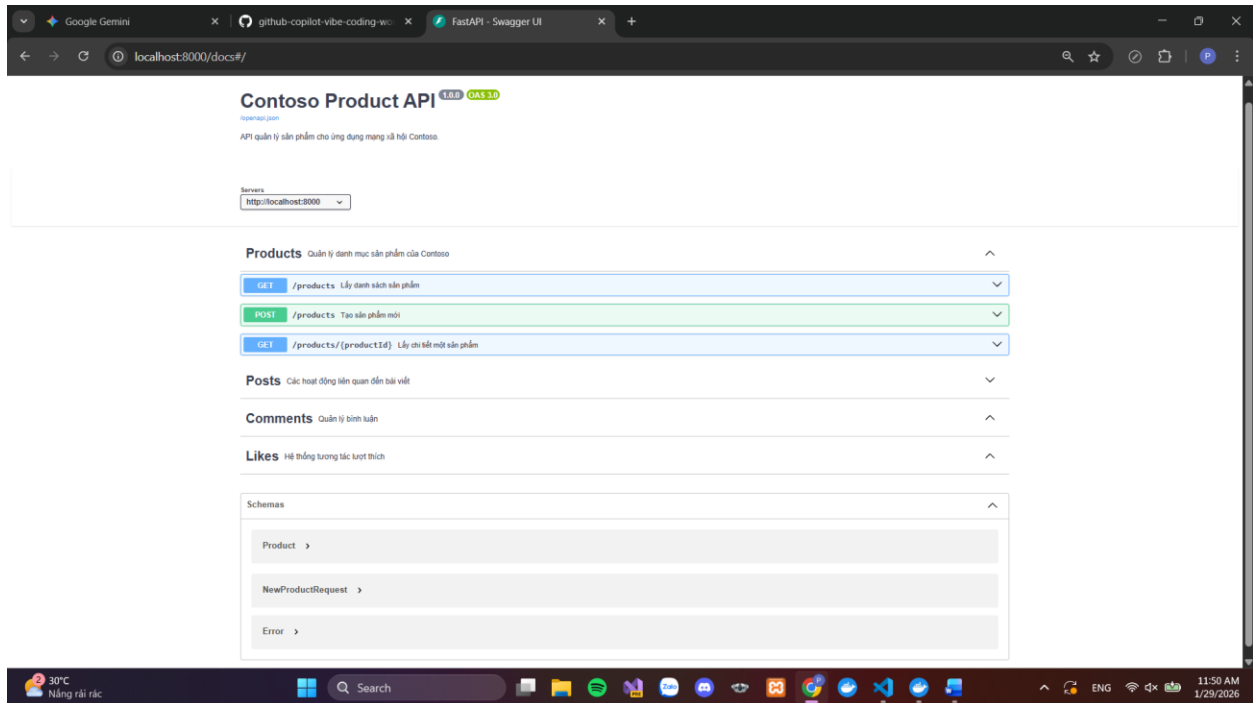
4.1. Xác minh và Kiểm tra (Verify App Running)

- **Thao tác:** Chạy server bằng lệnh:

python -m uvicorn main:app --reload

Kiểm tra: Truy cập <http://localhost:8000/docs> trên trình duyệt.

Xử lý sự cố: Đã điều chỉnh tệp openapi.yaml để khớp các tham chiếu \$ref và Schema Error, giúp trang tài liệu hiển thị sạch lỗi đỏ và đầy đủ mục **Products**.



02. JavaScript Frontend Development

2.1. Phương pháp luận (Methodology)

Trong giai đoạn này, dự án áp dụng phương pháp Vibe Coding – một quy trình phát triển phần mềm hiện đại dựa trên sự hỗ trợ của trí tuệ nhân tạo (GitHub Copilot) để tối ưu hóa hiệu suất từ khâu thiết kế đến thực thi.

- Design-to-Code Conversion: Chuyển đổi trực tiếp các lớp (layers) và thuộc tính từ bản thiết kế Figma (màu sắc, khoảng cách, kiểu chữ) thành các thành phần mã nguồn React tương ứng.
- Component-Driven Architecture: Xây dựng ứng dụng dựa trên các thành phần độc lập (Components), giúp mã nguồn dễ đọc, dễ bảo trì và mở rộng trong tương lai.
- API-Driven Integration: Giao diện được thiết kế để tiêu thụ (consume) dữ liệu từ các Endpoint RESTful đã xây dựng ở Mục 01 thông qua định dạng JSON.

2.2. Kiến trúc hệ thống (System Architecture)

Dự án vận hành theo mô hình Client-Server tách biệt, đảm bảo tính linh hoạt cao:

- Frontend (Client): Sử dụng React.js phối hợp với Vite để đảm bảo tốc độ phản hồi giao diện cực nhanh và trải nghiệm người dùng mượt mà.
- Backend (Server): Sử dụng FastAPI (Python) để xử lý các yêu cầu nghiệp vụ và truy vấn cơ sở dữ liệu.

- Communication: Giao thức HTTP/HTTPS được sử dụng để trao đổi dữ liệu không đồng bộ (Asynchronous) thông qua hàm fetch.

2.3. Quy trình thực hiện chi tiết

Bước 1: Khởi tạo và Cấu hình môi trường

- Thiết lập dự án React bằng công cụ Vite để tối ưu hóa quá trình phát triển.
- Cài đặt các thư viện hỗ trợ quan trọng: lucide-react (để hiển thị icon hệ thống) và cấu hình cổng truy cập (Port 5173/3000).

Bước 2: Hiện thực hóa giao diện (UI Implementation)

- Sử dụng GitHub Copilot để sinh mã cho các tệp cấu trúc chính như App.jsx.
- Tích hợp logic xử lý sự kiện cho nút "Đăng bài" (Handle Submit) và quản lý trạng thái nhập liệu (State Management).

Bước 3: Kết nối và Xử lý dữ liệu liên thông

- Sử dụng Hook useEffect để tự động truy xuất danh sách bài đăng từ Backend ngay khi trang web được tải.
- Cấu hình cơ chế CORS (Cross-Origin Resource Sharing) trên Python để cho phép Frontend truy cập tài nguyên an toàn.

03. Java Migration from Python (FastAPI → Spring Boot)

3.1. Scenario

Contoso là một công ty kinh doanh các sản phẩm phục vụ hoạt động ngoài trời. Bộ phận marketing của công ty mong muốn xây dựng một website mạng xã hội quy mô nhỏ (micro social media website) nhằm quảng bá sản phẩm tới khách hàng hiện tại và tiềm năng.

Hệ thống backend ban đầu được xây dựng bằng Python với framework FastAPI. Tuy nhiên, do lập trình viên Python đã rời công ty, các bên liên quan quyết định **di chuyển (migrate) toàn bộ backend API sang Java**, sử dụng **Spring Boot**, nhằm phù hợp với năng lực đội ngũ hiện tại.

Trong bối cảnh đó, vai trò của chúng tôi là **lập trình viên Java**, với hiểu biết hạn chế về Python/FastAPI, thực hiện việc migration một cách chính xác, an toàn và có kiểm soát.

3.2. Mục tiêu và ràng buộc

3.2.1. Mục tiêu

- Di chuyển toàn bộ backend API từ FastAPI (Python) sang Spring Boot (Java).
- Đảm bảo **tính tương đương chức năng** giữa hai hệ thống.
- Tuân thủ kiến trúc và chuẩn thực hành của Spring Boot.
- Đảm bảo hệ thống mới có thể build và chạy ổn định.

3.2.2. Ràng buộc chính

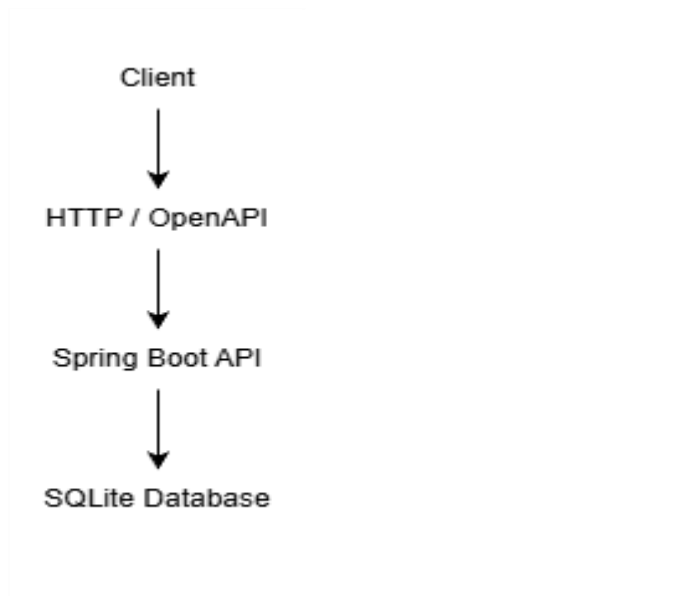
- **openapi.yaml là nguồn chân lý duy nhất (single source of truth).**
- Không được thêm, bớt hay chỉnh sửa bất kỳ endpoint, HTTP method, hay schema nào không được định nghĩa trong OpenAPI.
- Công nghệ sử dụng:
 - Java 21
 - Spring Boot
 - Gradle
 - SQLite (sns_api.db)
- Cơ sở dữ liệu phải được **khởi tạo mỗi khi ứng dụng khởi động.**
- Áp dụng **kiến trúc phân tầng**: Controller → Service → Repository.

3.3. Phương pháp luận Migration

Quá trình migration được thực hiện theo hướng **top-down**, bắt đầu từ mức khái niệm cho đến mức triển khai vật lý, nhằm giảm thiểu rủi ro và đảm bảo khả năng kiểm soát.

3.3.1. Mức ý tưởng (Conceptual Level)

Ở mức này, hệ thống được nhìn nhận như một khối chức năng tổng thể, tập trung vào mối quan hệ giữa các thành phần chính: client, backend API và cơ sở dữ liệu.

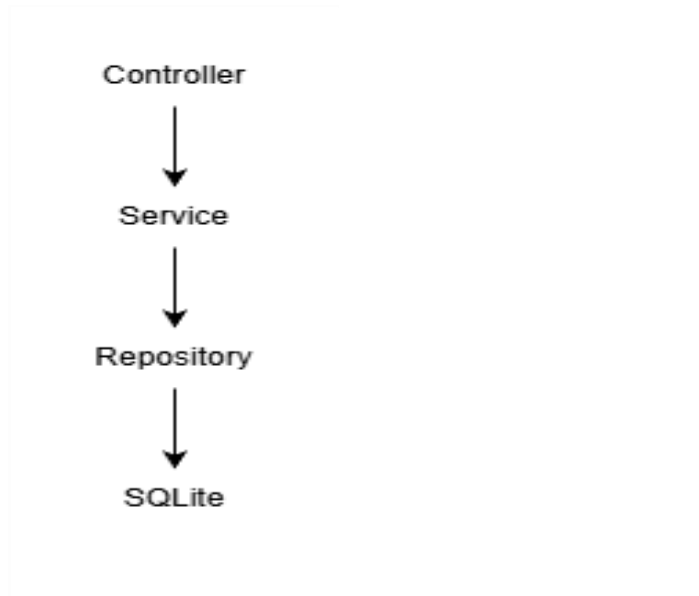


Hình 3.1 Kiến trúc tổng quan ở mức ý tưởng

Hình trên cho thấy backend API đóng vai trò trung tâm, giao tiếp với client thông qua HTTP dựa trên hợp đồng OpenAPI, và tương tác với cơ sở dữ liệu SQLite để lưu trữ dữ liệu.

3.3.2. Mức luận lý (Logical / Architectural Level)

Ở mức luận lý, hệ thống được thiết kế theo **kiến trúc phân tầng chuẩn của Spring Boot**, giúp tách biệt rõ ràng trách nhiệm của từng thành phần.



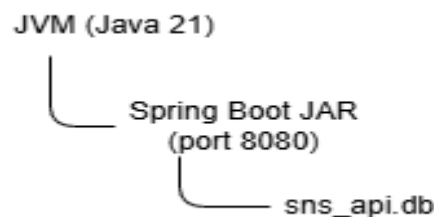
Hình 3.2 Kiến trúc phân tầng của Spring Boot Backend

- **Controller Layer:** Tiếp nhận HTTP request, ánh xạ chính xác các endpoint được định nghĩa trong openapi.yaml.
- **Service Layer:** Chứa logic nghiệp vụ, đảm bảo hành vi tương đương với FastAPI.
- **Repository Layer:** Truy cập dữ liệu SQLite thông qua JDBC/JPA.

Cách tiếp cận này giúp đảm bảo khả năng mở rộng, bảo trì và kiểm thử hệ thống.

3.3.3. Mức vật lý (Physical / Deployment Level)

Mức vật lý mô tả cách hệ thống được triển khai và vận hành trên môi trường thực tế.



Hình 3.3 Triển khai vật lý của Spring Boot Application

Ứng dụng Spring Boot được đóng gói dưới dạng file JAR, chạy trên JVM với Java 21, lắng nghe tại cổng 8080. Cơ sở dữ liệu SQLite (sns_api.db) được lưu trữ cục bộ và được khởi tạo tự động khi ứng dụng khởi động.

3.4. Chuẩn bị môi trường (Prepare Phase)

3.4.1. Chuẩn bị GitHub Copilot Agent Mode

GitHub Copilot được sử dụng ở chế độ **Agent Mode** với mô hình GPT-4.1 hoặc Claude Sonnet 4, nhằm hỗ trợ phân tích, sinh mã và kiểm tra lỗi trong quá trình migration.

3.4.2. Chuẩn bị Custom Instructions

Các chỉ dẫn tùy chỉnh được cấu hình để đảm bảo Copilot:

- Nhận đúng vai trò lập trình viên Java
- Tuân thủ ràng buộc OpenAPI
- Áp dụng kiến trúc phân tầng

Việc chuẩn bị này được xác minh bằng cách yêu cầu Copilot tóm tắt vai trò và ràng buộc, đảm bảo các chỉ dẫn đã được áp dụng chính xác.

3.5. Chuẩn bị Spring Boot Project

Dự án Spring Boot được khởi tạo bằng **Spring Boot CLI** với các thông số:

- Group ID: com.contoso
- Artifact ID: socialapp
- Package: com.contoso.socialapp
- Java version: 21
- Dependencies:
 - Spring Web
 - Spring Boot Actuator
 - Lombok
- Build tool: Gradle
- Port: 8080
- CORS: cho phép tất cả nguồn

Sau khi khởi tạo, dự án được build và chạy thử để đảm bảo môi trường sẵn sàng cho bước migration.

3.6. Migration FastAPI sang Spring Boot

Quá trình migration được thực hiện dựa hoàn toàn trên đặc tả openapi.yaml:

- Phân tích toàn bộ endpoint, method và schema.
- Tạo Controller tương ứng trong Spring Boot.
- Ánh xạ logic nghiệp vụ sang Service.
- Tái sử dụng SQLite làm cơ sở dữ liệu.
- Cấu hình Swagger UI và OpenAPI endpoint hiển thị đúng nội dung mô tả.

Trong quá trình này, **không có bất kỳ chức năng hay API nào được thêm hoặc chỉnh sửa ngoài OpenAPI.**

3.7. Kiểm tra và xác minh

Sau khi migration hoàn tất:

- Ứng dụng được build thành công bằng Gradle.
- Ứng dụng chạy ổn định trên cổng 8080.
- Tất cả endpoint hoạt động đúng như mô tả trong openapi.yaml.
- Swagger UI hiển thị đầy đủ và chính xác tài liệu OpenAPI.

3.8. Kết luận

Việc di chuyển backend từ FastAPI sang Spring Boot đã được thực hiện thành công theo phương pháp luận rõ ràng, tuân thủ nghiêm ngặt đặc tả OpenAPI và chuẩn kiến trúc Spring Boot. Kết quả đảm bảo hệ thống mới có tính tương đương chức năng, dễ bảo trì và phù hợp với năng lực đội ngũ phát triển Java hiện tại.

04. .NET Migration from JavaScript (React → Blazor)

4.1. Scenario

Contoso là một công ty kinh doanh các sản phẩm phục vụ các hoạt động ngoài trời. Nhằm quảng bá sản phẩm tới khách hàng hiện tại và tiềm năng, công ty đã xây dựng một ứng dụng frontend cho website mạng xã hội nhỏ (micro social media website).

Ứng dụng frontend ban đầu được phát triển bằng JavaScript, sử dụng framework React. Tuy nhiên, do yêu cầu mới từ phía doanh nghiệp, hệ thống frontend cần được tái phát triển (migrate) sang nền tảng .NET, cụ thể là Blazor, nhằm đồng bộ công nghệ với backend và chiến lược phát triển lâu dài.

Trong bối cảnh này, vai trò của chúng tôi là lập trình viên .NET, với hiểu biết hạn chế về JavaScript và React, thực hiện quá trình migration từ React sang Blazor một cách có hệ thống và kiểm soát.

4.2. Mục tiêu và ràng buộc

4.2.1. Mục tiêu

- Di chuyển toàn bộ giao diện frontend từ React sang Blazor.

- Đảm bảo giao diện và hành vi của ứng dụng Blazor tương đương với ứng dụng React ban đầu.
- Đảm bảo frontend Blazor giao tiếp đúng với backend Spring Boot đã được xây dựng ở bước 03.
- Ứng dụng Blazor có thể build và chạy ổn định.

4.2.2. Ràng buộc chính

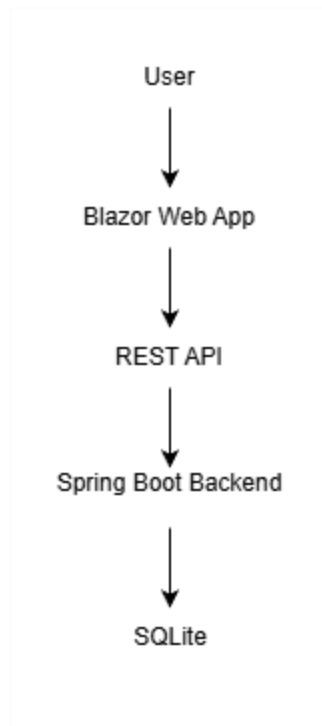
- Không thay đổi logic nghiệp vụ phía backend.
- Chỉ thực hiện migration frontend, không bổ sung chức năng mới.
- Công nghệ sử dụng:
 - .NET 9
 - Blazor
 - NuGet
- Backend API chạy tại **http://localhost:8080**.
- Ưu tiên sử dụng tính năng native của Blazor, chỉ sử dụng JavaScript khi thực sự cần thiết thông qua JSInterop.

4.3. Phương pháp luận Migration

Quá trình migration được thực hiện theo phương pháp ba mức độ trừu tượng, nhằm đảm bảo tính nhất quán và dễ kiểm soát trong quá trình chuyển đổi công nghệ.

4.3.1. Mức ý tưởng (Conceptual Level)

Ở mức ý tưởng, hệ thống được xem xét như một tổng thể gồm frontend Blazor, backend Spring Boot và người dùng cuối.

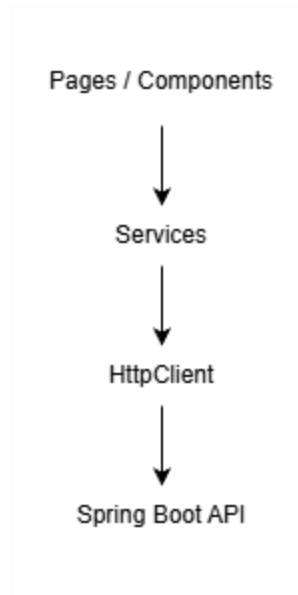


Hình 4.1 Kiến trúc tổng quan Frontend – Backend

Frontend Blazor giao tiếp với backend Spring Boot thông qua các REST API đã được định nghĩa sẵn, đảm bảo không phụ thuộc vào công nghệ frontend cụ thể.

4.3.2. Mức luận lý (Logical / Architectural Level)

Ở mức luận lý, kiến trúc nội bộ của frontend Blazor được tổ chức dựa trên mô hình component-based, tương đương với React.



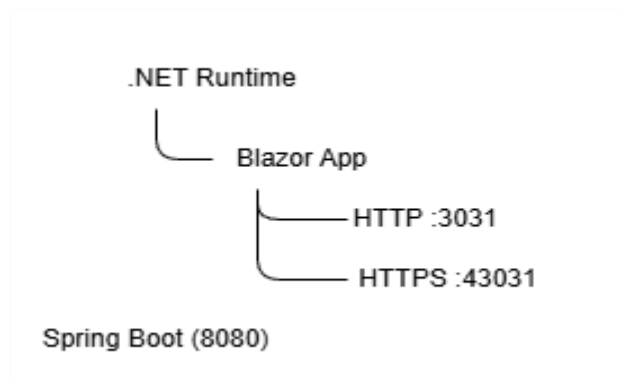
Hình 4.2 Kiến trúc component của Blazor Web App

- **Blazor Components:** Thay thế cho React components.
- **Pages:** Đại diện cho các route chính của ứng dụng.
- **Services:** Đảm nhiệm việc gọi HTTP API tới backend.
- **JSInterop (nếu cần):** Kết nối JavaScript với Blazor trong các trường hợp đặc biệt.

Kiến trúc này giúp việc ánh xạ từ React sang Blazor trở nên trực quan và có hệ thống.

4.3.3. Mức vật lý (Physical / Deployment Level)

Mức vật lý mô tả cách ứng dụng Blazor được build và chạy trên môi trường thực tế.



Hình 4.3 Triển khai vật lý của Blazor Web Application

Ứng dụng Blazor chạy trên nền .NET, lắng nghe tại cổng 3031 (HTTP) và 43031 (HTTPS), giao tiếp với backend Spring Boot đang chạy trên cổng 8080.

4.4. Chuẩn bị môi trường (Prepare Phase)

4.4.1. Chuẩn bị GitHub Copilot Agent Mode

GitHub Copilot được sử dụng ở chế độ Agent Mode với mô hình GPT-4.1 hoặc Claude Sonnet 4 nhằm hỗ trợ phân tích cấu trúc React, sinh mã Blazor và xử lý lỗi trong quá trình migration.

4.4.2. Chuẩn bị Custom Instructions

Các chỉ dẫn tùy chỉnh được cấu hình để đảm bảo Copilot:

- Nhận đúng vai trò lập trình viên .NET
- Hiểu rõ nhiệm vụ migration React → Blazor
- Tuân thủ kiến trúc Blazor và chuẩn .NET

Việc chuẩn bị này được xác minh bằng cách yêu cầu Copilot tóm tắt lại vai trò và các ràng buộc của dự án.

4.5. Chuẩn bị Blazor Web App Project

Dự án Blazor được khởi tạo với các đặc điểm sau:

- Project name: Contoso.BlazorApp
- Solution name: ContosoWebApp
- Nền tảng: Blazor Web App
- HTTP port: 3031
- HTTPS port: 43031

Sau khi tạo project, ứng dụng được build và chạy thử để đảm bảo môi trường phát triển sẵn sàng cho bước migration.

4.6. Migration React sang Blazor

Quá trình migration được thực hiện theo các bước:

- Phân tích cấu trúc thư mục và các component của ứng dụng React.

- Ánh xạ từng React component sang Blazor component tương ứng.
- Chuyển đổi logic xử lý giao diện từ JavaScript sang C#.
- Di chuyển và tái sử dụng các file CSS cần thiết.
- Sử dụng HttpClient trong Blazor để giao tiếp với backend Spring Boot.
- Áp dụng JSInterop trong các trường hợp không thể thay thế hoàn toàn JavaScript.

Mục tiêu của bước này là đảm bảo ứng dụng Blazor có hành vi và giao diện gần nhất với ứng dụng React ban đầu.

4.7. Kiểm tra và xác minh

Sau khi hoàn tất migration:

- Ứng dụng Blazor được build thành công.
- Ứng dụng chạy ổn định tại <http://localhost:3031>.
- Giao diện hiển thị đúng và có thể tương tác.
- Kết nối tới backend Spring Boot hoạt động chính xác.

4.8. Kết luận

Quá trình di chuyển frontend từ React sang Blazor đã được thực hiện thành công theo phương pháp luận rõ ràng, đảm bảo tính tương đương chức năng và tuân thủ chuẩn phát triển .NET. Kết quả cho thấy Blazor là một nền tảng phù hợp để thay thế React trong bối cảnh hệ sinh thái .NET, đồng thời đảm bảo khả năng tích hợp chặt chẽ với backend Spring Boot.

05. Containerization (Java Backend & .NET Frontend)

5.1. Scenario

Sau khi hoàn thành việc migration backend từ FastAPI sang Spring Boot (mục 03) và frontend từ React sang Blazor (mục 04), hệ thống của Contoso hiện bao gồm:

- Backend: Ứng dụng Spring Boot (Java 21)
- Frontend: Ứng dụng Blazor (.NET 9)

Để đảm bảo khả năng **triển khai linh hoạt, nhất quán trên nhiều môi trường** (local, cloud, CI/CD), doanh nghiệp yêu cầu **container hóa toàn bộ hệ thống**.

Trong giai đoạn này, vai trò của chúng tôi là **DevOps engineer**, chịu trách nhiệm container hóa các ứng dụng và tổ chức chúng chạy đồng thời thông qua cơ chế orchestration.

5.2. Mục tiêu và ràng buộc

5.2.1. Mục tiêu

- Container hóa backend Spring Boot và frontend Blazor.
- Đảm bảo mỗi ứng dụng có thể chạy độc lập trong container.
- Cho phép hai container giao tiếp với nhau khi được orchestration.
- Chuẩn bị nền tảng cho việc triển khai trên các hệ thống container (Docker, Kubernetes).

5.2.2. Ràng buộc chính

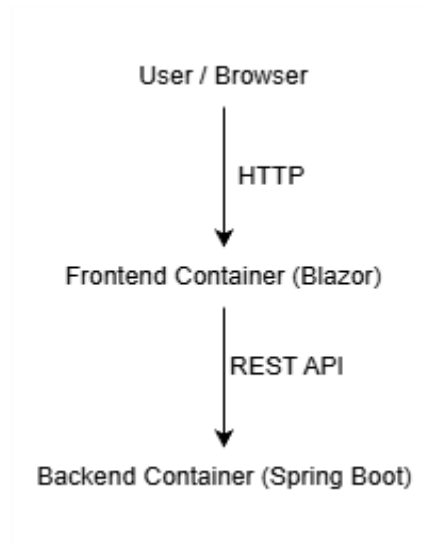
- Sử dụng Docker và Docker Compose.
- Áp dụng **multi-stage build** cho cả Java và .NET.
- Không sao chép file database SQLite từ máy host vào container.
- Tuân thủ đúng cấu hình cổng mạng được yêu cầu.
- Không thay đổi logic ứng dụng backend và frontend.

5.3. Phương pháp luận Containerization

Quá trình containerization được tiếp cận theo **ba mức độ trừu tượng**, tương tự như các bước migration trước đó, nhằm đảm bảo tính hệ thống và khả năng mở rộng.

5.3.1. Mức ý tưởng (Conceptual Level)

Ở mức ý tưởng, hệ thống được xem như một tập hợp các dịch vụ độc lập, mỗi dịch vụ được đóng gói trong một container riêng.

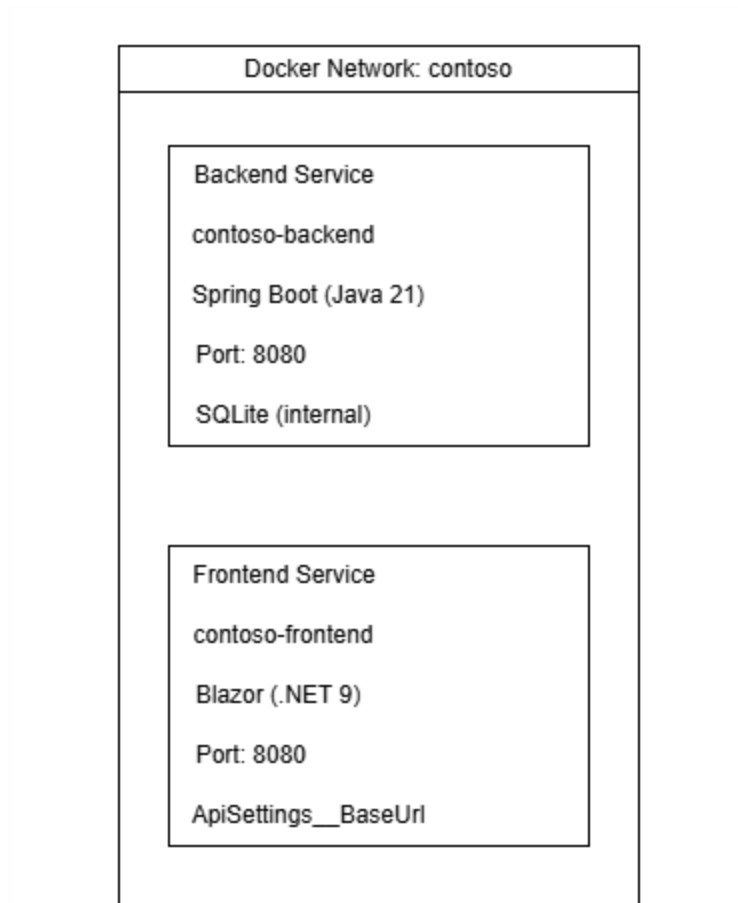


Hình 5.1 Kiến trúc container hóa tổng quan

Người dùng truy cập frontend container, frontend giao tiếp với backend container thông qua mạng Docker nội bộ.

5.3.2. Mức luận lý (Logical / Architectural Level)

Ở mức luận lý, hệ thống được chia thành hai service chính và được quản lý thông qua Docker Compose.



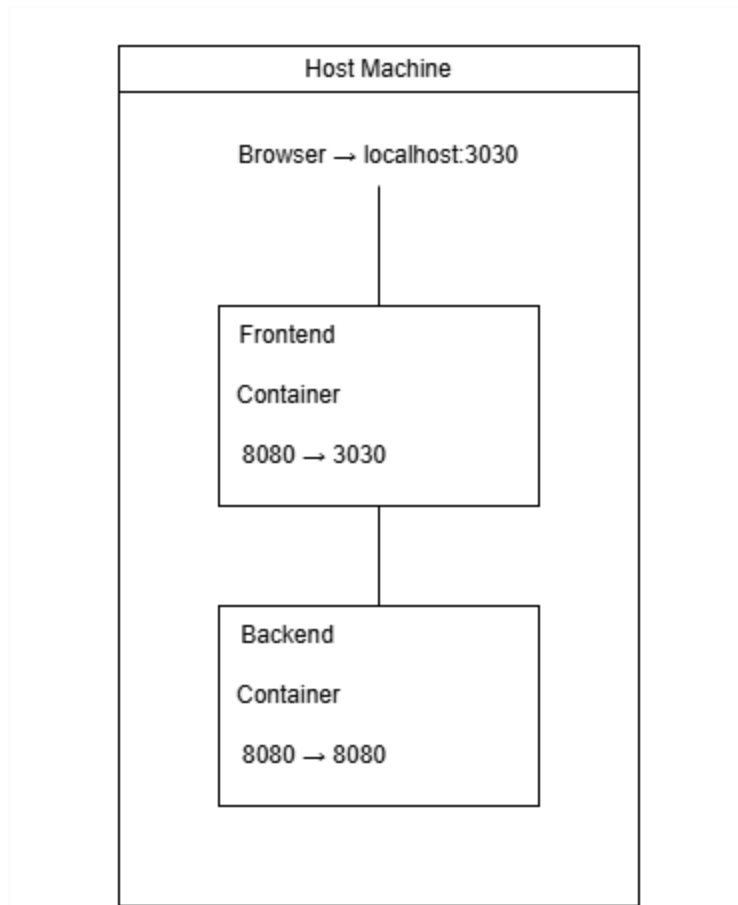
Hình 5.2 Kiến trúc logic các container

- **Backend Service:**
 - Image: contoso-backend:latest
 - Công nghệ: Spring Boot (Java 21)
 - Database: SQLite (nội bộ container)
- **Frontend Service:**
 - Image: contoso-frontend:latest
 - Công nghệ: Blazor (.NET 9)
 - Giao tiếp backend thông qua biến môi trường

Các service được kết nối thông qua một network chung trong Docker.

5.3.3. Mức vật lý (Physical / Deployment Level)

Mức vật lý mô tả cách các container được build, chạy và expose cổng ra môi trường host.



Hình 5.3 Triển khai vật lý với Docker Compose

- Backend container:
 - Target port: 8080
 - Host port: 8080
- Frontend container:
 - Target port: 8080
 - Host port: 3030

5.4. Chuẩn bị môi trường (Prepare Phase)

5.4.1. Chuẩn bị GitHub Copilot Agent Mode

GitHub Copilot được sử dụng ở chế độ **Agent Mode** với mô hình GPT-4.1 hoặc Claude Sonnet 4 nhằm hỗ trợ sinh Dockerfile, Docker Compose và xử lý lỗi trong quá trình build image.

5.4.2. Chuẩn bị Custom Instructions

Các custom instructions được cấu hình để đảm bảo Copilot:

- Nhận đúng vai trò DevOps engineer
- Tuân thủ yêu cầu container hóa
- Không thay đổi logic ứng dụng

Việc chuẩn bị được xác minh bằng cách yêu cầu Copilot tóm tắt vai trò và các ràng buộc của giai đoạn containerization.

5.5. Container hóa Java Backend

Ứng dụng Spring Boot được container hóa với các đặc điểm:

- Dockerfile: Dockerfile.java
- Sử dụng Microsoft OpenJDK 21
- Áp dụng multi-stage build
- Tách JRE từ JDK để giảm kích thước image
- Tạo mới file SQLite sns_api.db trong container khi build

Sau khi build thành công, image backend được đặt tên là contoso-backend:latest và được kiểm tra bằng cách chạy container độc lập trên cổng 8080.

5.6. Container hóa .NET Frontend

Ứng dụng Blazor được container hóa với các đặc điểm:

- Dockerfile: Dockerfile.dotnet
- Nền tảng: .NET 9
- Áp dụng multi-stage build
- Cấu hình biến môi trường ApiSettings__BaseUrl để trỏ tới backend API

Image frontend được đặt tên là contoso-frontend:latest và được kiểm tra bằng cách chạy container độc lập trên cổng 3030.

5.7. Orchestration với Docker Compose

Sau khi hai image được build và kiểm tra độc lập, hệ thống được orchestration bằng Docker Compose.

- File: compose.yaml
- Network: contoso
- Hai service: backend và frontend
- Cấu hình biến môi trường cho từng container

Docker Compose cho phép khởi động đồng thời hai ứng dụng và thiết lập kết nối mạng nội bộ giữa chúng.

5.8. Kiểm tra và xác minh

Sau khi chạy Docker Compose:

- Cả hai container đều khởi động thành công.
- Frontend truy cập được tại <http://localhost:3030>.
- Frontend giao tiếp đúng với backend thông qua network Docker.
- Backend API hoạt động ổn định trong container.

5.9. Kết luận

Quá trình container hóa hệ thống đã được thực hiện thành công, giúp đóng gói backend và frontend thành các đơn vị triển khai độc lập, nhất quán và có khả năng mở rộng. Việc sử dụng Docker và Docker Compose tạo tiền đề cho các bước triển khai nâng cao hơn như CI/CD hoặc Kubernetes trong tương lai.